Writing recipes - advanced



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Extending a recipe



Introduction to recipe extensions

- It is a good practice to avoid modifying recipes available in third party layers so it is easy to update.
- But it is sometimes useful to apply a custom patch or add a configuration file for example.
- The `bitbake` *build engine* allows to modify a recipe by extending it.
- Multiple extensions can be applied to a recipe.

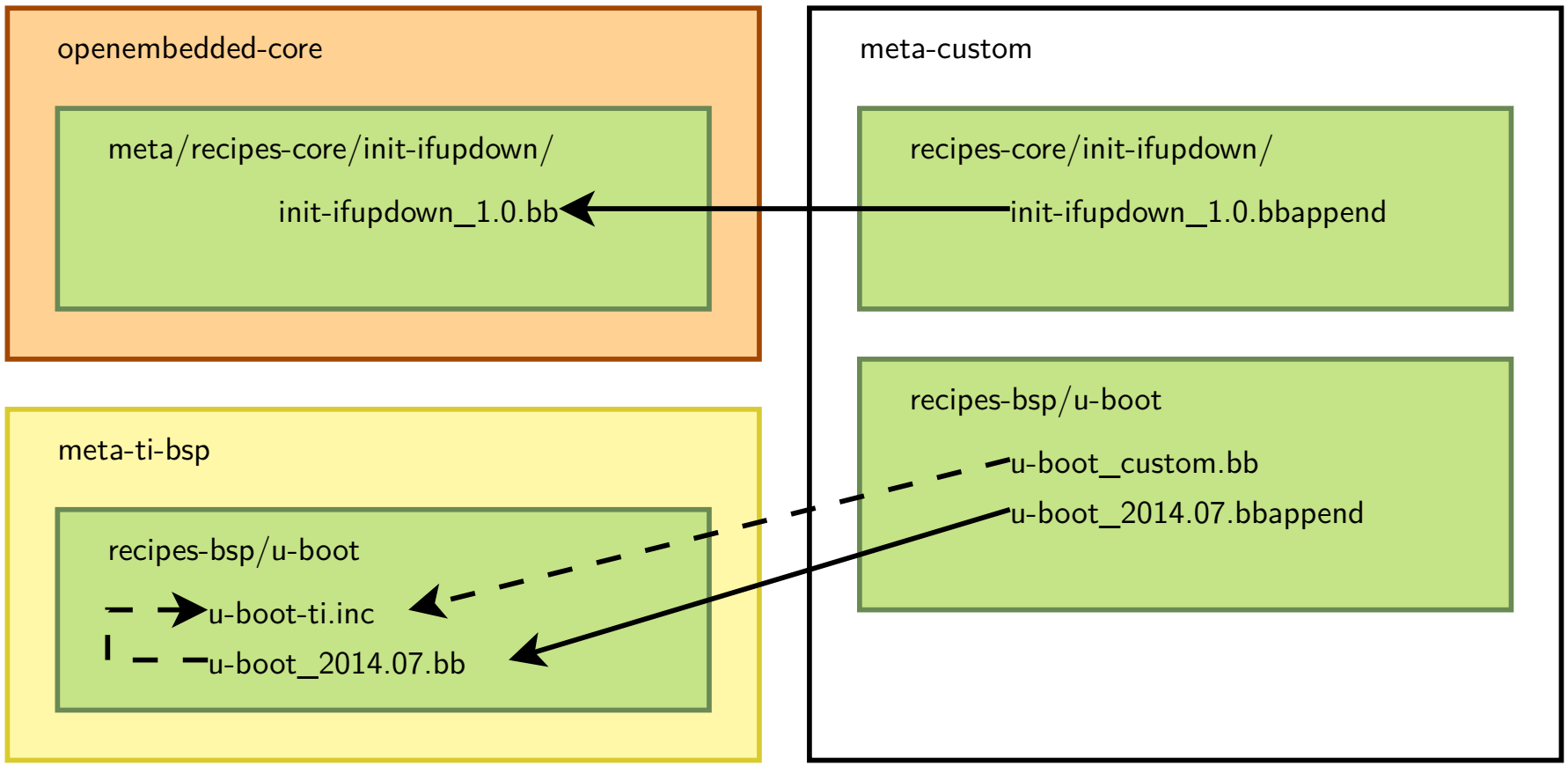


Introduction to recipe extensions

- Variable values can be modified
  - Using operators to set, append or prepend
  - Using overrides to append, prepend or remove
- Tasks can be modified
  - Using overrides to append or prepend task code
  - Adding new tasks



Extend a recipe





Extend a recipe

- The recipe extensions end in `.bbappend`
- Append files must have the same root name as the recipe they extend, but can also use wildcards.
  - `example_0.1.bbappend` applies to `example_0.1.bb`
  - `example_0.%.bbappend` applies to `example_0.2.bb` but not `example_1.0.bb`
  - The % works only just before the `.bbappend` suffix
- Append files should be **version specific**. If the recipe is updated to a newer version, the append files must also be updated.
- If adding new files, the path to their directory must be prepended to the FILESEXTRAPATHS variable.
  - Files are looked up in paths referenced in FILESEXTRAPATHS, from left to right.
  - Prepending a path makes sure it has priority over the recipe's one. This allows to override recipes' files.



Append file example

`linux-yocto_6.12.bbappend`:

```
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"

SRC_URI += "file://defconfig \
            file://fix-memory-leak.patch \
            "
```

File organization:

```
meta-custom/
`-- recipes-kernel/
  `-- linux/
        |-- files/
        |   |-- defconfig
        |   `-- fix-memory-leak.patch
        `-- linux-yocto_6.12.bbappend
```



Modifying existing tasks

Tasks can be extended with `:prepend` or `:append`

```
do_install:append() {
    install -d ${D}${sysconfdir}
    install -m 0644 hello.conf ${D}${sysconfdir}
}
```

Overrides can also be used for extending tasks:

```
do_install:append:beaglebone() {
    install -d ${D}${nonarch_base_libdir}/firmware
    install -m 0644 firmware.bin ${D}${nonarch_base_libdir}/firmware
}
```



Virtual providers (cont.)



Virtual providers (cont.)

- `bitbake` allows to use virtual names instead of the actual recipe name. We saw a use case with *virtual providers*.
- The virtual name is specified through the PROVIDES variable.
- Several recipes can provide the same virtual name. Only one will be built and installed into the generated image.
- `PROVIDES += "virtual/kernel"`



Classes



Introduction to classes

- Classes provide an abstraction to common code, which can be reused in multiple recipes.
- Common tasks do not have to be re-developed!
- Any metadata and task which can be put in a recipe can be used in a class.
- Classes extension is `.bbclass`
- Classes must be located in the `classes-recipe`, `classes-global`, or `classes` folders of a layer.



Using classes

- Recipes can use the classes located in the `classes-recipe` folder:
  - `inherit class1 class2 ...`
- A recipe can inherit from multiple classes.
- Classes in `classes-global` can be inherited from configuration files with INHERIT:
  - `INHERIT:append = " class1 class2"`
- Classes in `INHERIT` will be used by every built recipe.
- Classes in the `classes` folder can be used with `inherit` or INHERIT, as their usage is not clearly defined.



Common classes

- classes-global/base.bbclass
- classes-recipe/kernel.bbclass
- classes-recipe/autotools.bbclass
- classes-recipe/autotools-brokensep.bbclass
- classes-recipe/cmake.bbclass
- classes-recipe/meson.bbclass
- classes-recipe/native.bbclass
- classes-recipe/systemd.bbclass
- classes-recipe/update-rc.d.bbclass
- classes/useradd.bbclass
- ...



The base class

- Every recipe inherits the `base` class automatically.
- Defines basic common tasks with a default implementation:
  - `fetch`, `unpack`, `patch`
  - `configure`, `compile`, `install`
  - Utility tasks such as: `clean`, `listtasks`
- Automatically applies patch files listed in `SRC_URI`
- Defines `oe_runmake`, using EXTRA_OEMAKE to use custom arguments.



The kernel class

- Used to build Linux kernels.
- Defines tasks to configure, compile and install a kernel and its modules.
- Automatically applies a `defconfig` listed in SRC_URI

```
SRC_URI += "file://defconfig"
```

- The kernel is divided into several packages: `kernel`, `kernel-base`, `kernel-dev`, `kernel-modules`...
- Automatically provides the virtual package `virtual/kernel`.
- Configuration variables are available:
  - KERNEL_IMAGETYPE, defaults to `zImage`
  - KERNEL_EXTRA_ARGS
  - INITRAMFS_IMAGE



The autotools class

- Defines tasks and metadata to handle applications using the autotools build system (autoconf, automake and libtool):
  - `do_configure`: generates the configure script using `autoreconf` and loads it with standard arguments or cross-compilation.
  - `do_compile`: runs `make`
  - `do_install`: runs `make install`
- Extra configuration parameters can be passed with EXTRA_OECONF.
- Compilation flags can be added thanks to the EXTRA_OEMAKE variable.



Example: use the autotools class

```
DESCRIPTION = "Print a friendly, customizable greeting"
HOMEPAGE = "https://www.gnu.org/software/hello/"
SECTION = "examples"
LICENSE = "GPL-3.0-or-later"

SRC_URI = "${GNU_MIRROR}/hello/hello-${PV}.tar.gz"
SRC_URI[sha256sum] = "ecbb7a2214196c57ff9340aa71458e1559abd38f6d8d169666846935df191ea7"
LIC_FILES_CHKSUM = "file://COPYING;md5=d32239bcb673463ab874e80d47fae504"

inherit autotools
```



The useradd class

- This class helps to add users to the resulting image.
- Adding custom users is required by many services to avoid running them as root.
- USERADD_PACKAGES must be defined when the `useradd` class is inherited. It defines the individual packages produced by the recipe that need users or groups to be added.
- Users and groups will be created before the packages using it perform their `do_install`.
- At least one of the two following variables must be set:
  - USERADD_PARAM: parameters to pass to `useradd`.
  - GROUPADD_PARAM: parameters to pass to `groupadd`.



Example: use the useradd class

```
DESCRIPTION = "useradd class usage example"
SECTION = "examples"
LICENSE = "MIT"

SRC_URI = "file://file0"
LIC_FILES_CHKSUM = "file://${COREBASE}/meta/files/common-licenses/MIT;md5=0835ade698e0bc..."

inherit useradd

USERADD_PACKAGES = "${PN}"
USERADD_PARAM:${PN} = "-u 1000 -d /home/user0 -s /bin/bash user0"

FILES:${PN} = "/home/user0/file0"

do_install() {
    install -d ${D}/home/user0/
    install -m 644 file0 ${D}/home/user0/
    chown user0:user0 ${D}/home/user0/file0
}
```



The bin_package class

- In some cases you only need to install pre-built files into the generated root filesystem
  - E.g.: firmware blobs
- `bin_package.bbclass` simplifies this
  - Disables `do_configure` and `do_compile`
  - Provides a default `do_install` that copies whatever is in `S` (useful e.g. when extracting a pre-built RPM/DEB)
- Additionally set the `LICENSE` to `CLOSED` if applicable



BitBake file inclusions



Locate files in the build system

- Metadata can be shared using included files.
- `BitBake` uses the BBPATH to find the files to be included. It also looks into the current directory.
- Three keywords can be used to include files from recipes, classes or other configuration files:
  - `inherit`
  - `include`
  - `require`



The `inherit` keyword

- `inherit` can be used in recipes or classes, to inherit the functionalities of a class.
- To inherit the functionalities of the `kernel` class, use: `inherit kernel`
- `inherit` looks for files ending in `.bbclass`, in `classes`, `classes-recipe` or `classes-global` directories found in BBPATH.
- It is possible to include a class conditionally using a variable: `inherit ${FOO}`
- Inheriting in configuration files is based on the INHERIT variable instead:
  - `INHERIT += "rm_work"`
  - This inherits the class globally (i.e. for all recipes)



The `include` and `require` keywords

- `include` and `require` can be used in all files, to insert the content of another file at that location.
- If the path specified on the `include` (or `require`) path is relative, `bitbake` will insert the first file found in BBPATH.
- `include` does not produce an error when a file cannot be found, whereas `require` raises a parsing error.
- To include a local file: `require ninvaders.inc`
- To include a file from another location (which could be in another layer):
  `require path/to/file.inc`



More recipe debugging tools



More recipe debugging tools

- A development shell, exporting the full environment can be used to debug build failures:

```
$ bitbake -c devshell <recipe>
```

- To understand what a change in a recipe implies, you can activate build history in `local.conf`:

```
INHERIT:append = " buildhistory"
```

Then use the `buildhistory-diff` tool to examine differences between two builds:

```
$ buildhistory-diff
Changes to .../core-image-minimal (files-in-image.txt):
  /usr/bin/bash was added
  ...
Changes to .../core-image-minimal (installed-package-names.txt):
  bash was added
  ...
```



Network usage



Source fetching

- `bitbake` will look for files to retrieve at the following locations, in order:
  1. DL_DIR (the local download directory).
  2. The PREMIRRORS locations.
  3. The upstream source, as defined in SRC_URI.
  4. The MIRRORS locations.
- If all the mirrors fail, the build will fail.



Mirror configuration in OpenEmbedded-Core

meta/classes-global/mirrors.bbclass

```
PREMIRRORS += "git://sourceware.org/git/glibc.git        https://downloads.yoctoproject.org/mirror/sources/ \
               git://sourceware.org/git/binutils-gdb.git https://downloads.yoctoproject.org/mirror/sources/"

MIRRORS += "
svn://.*/.*     http://downloads.yoctoproject.org/mirror/sources/ \
git://.*/.*     http://downloads.yoctoproject.org/mirror/sources/ \
https?://.*/.*  http://downloads.yoctoproject.org/mirror/sources/ \
ftp://.*/.*     http://downloads.yoctoproject.org/mirror/sources/ \
...
"
```



Configuring the premirrors

- It is easy to add a custom mirror to the PREMIRRORS by using the `own-mirrors` class (only one URL supported):

```
INHERIT += "own-mirrors"
SOURCE_MIRROR_URL = "http://example.com/my-mirror"
```

- For a more complex setup, prepend custom mirrors to the PREMIRRORS variable:

```
PREMIRRORS:prepend = "\
 git://.*/.*   http://example.com/my-mirror-for-git/ \
 svn://.*/.*   http://example.com/my-mirror-for-svn/ \
 http://.*/.*  http://www.yoctoproject.org/sources/  \
 https://.*/.* http://www.yoctoproject.org/sources/  "
```



Creating a local mirror

- The download directory can be exposed on the network to create a local mirror
  - Except for sources fetched via an SCM a tarball of the repository is needed, not the bare git repository that is created by default
  - You can use BB_GENERATE_MIRROR_TARBALLS = "1" to generate tarballs of the git repositories in DL_DIR



Forbidding network access

- Since Kirkstone (4.0), network access is only enabled in the `do_fetch` task, to make sure no untraced sources are fetched.
- You can also completely disable network access using BB_NO_NETWORK = "1"
  - To download all the sources before disabling network access use `bitbake --runall=fetch core-image-minimal`
- Or restrict `bitbake` to only download files from the PREMIRRORS, using BB_FETCH_PREMIRRORONLY = "1"